

WEEK 11 — QUESTION 1

Multithreaded Counter and Race Condition

Question

Develop a multithreaded counter application using POSIX threads (`pthread_create`, `pthread_join`). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Concepts

`pthread_create()`

`pthread_join()`

shared variable

race condition

multiple threads

```
        return NULL;
    }

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Initial counter = %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i],
                      NULL,
                      increment_counter,
                      NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter = %d\n", counter);

    printf("Expected counter = %d\n",
          NUM_THREADS * ITERATIONS);

    return 0;
}
saikrishna__90156@DESKTOP-2JF9H5T:~$
```

```
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano Practical11.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc Practical11.c -o Practical11
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Practical11
Initial counter = 0
Final counter = 206152
Expected counter = 400000
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat Practical11.c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Initial counter = %d\n", counter);
```